

第8章 网络通信

建议学时:4–6 小时 | **难度:**★★★★☆ | **读者背景:**熟悉 C 的 socket 编程(socket/bind/listen/accept/connect/send/recv)与基本的 TCP/UDP 概念

本章学习目标

- 掌握 std::net 的 TcpListener/TcpStream 与 C 的 socket 三件套的完整对应关系
- 掌握客户端 connect 与服务端 bind/listen/accept 循环的 Rust 写法,错误全程 Result
- 理解 TCP 是字节流而非消息流: 粘包/拆包的本质 与行协议、长度前缀两种解法
- 掌握读写超时与半关闭(shutdown),对照 C 的 SO_RCVTIMEO 与 shutdown(2)
- 掌握 UDP 的 send_to/recv_from,对照 sendto/recvfrom 与无连接语义
- 会用 request 写 HTTP 客户端,理解阻塞客户端与异步客户端的选择
- 能手写一个 ≤40 行的最小 HTTP 服务端,读懂请求行与响应格式
- 理解 TIME_WAIT 与端口复用在生产中的意义,以及 Rust 标准库的对应立场

从 C 到 Rust:本章主题如何迁移

C 的网络编程是"初始化仪式 + 全局错误"的双重折磨:WSAStartup(Windows)/socket() 后还要 bind、listen、accept 一步步来,每步都检查负数返回值;错误码藏在 errno/WSAGetLastError 里;还得自己管理 fd 生命周期,close 漏了就 fd 泄漏。Rust 的 std::net 把这些压扁成三个类型:TcpListener(监听)、TcpStream(连接)、UdpSocket(数据报)。bind 与 listen 合为一个构造器,TcpListener::bind 返回 Result;accept 返回 (TcpStream, SocketAddr)。套接字是 RAII:离开作用域自动关闭,不存在 fd 泄漏。

但网络编程的真正难点与语言无关,反而是字节流语义:TCP 没有消息边界,你在 C 里遇到的"一次 recv 只收到半条消息""两条消息粘在一起",在 Rust 里一模一样——因为这是 TCP 的物理事实,不是 API 缺陷。Rust 能帮你的,是提供 BufReader/BufWriter 与类型化的流读写,把"按行读网络数据"写成与读文件相同的样子(第7章的知识直接复用)。本章的策略:先建立 API 对照(\$8.1–\$8.3),再攻克字节流语义(\$8.5),然后 UDP 与 HTTP(\$8.6–\$8.7)——实验做成回声服务器,亲手验证粘包与拆包。

前置知识自查

- C 的 socket 编程:socket/bind/listen/accept/connect/send/recv/close,阻塞 IO 语义
- TCP 与 UDP 的区别:连接 vs 无连接、字节流 vs 数据报、可靠性
- 第5章的线程与所有权(本章每连接一线程要用 move 闭包)
- 第7章的 BufReader/Write 与 ? 错误传播(网络读写与文件读写同一套接口)
- HTTP 基本结构:请求行/请求头/空行/正文;状态码 200/404

本章学习路线

1. **第一阶段(约1.5小时):**§8.1–§8.3,客户端与服务端的骨架——先跑通"连上、发一句、收一句"
2. **第二阶段(约1.5小时):**§8.4–§8.5,超时与字节流边界——网络编程真正的生产门槛,粘包拆包
3. **第三阶段(约1.5小时):**§8.6–§8.7,UDP 与 HTTP——覆盖生产网络编程的两个高频场景
4. **验收标准:**能徒手写多客户端回声服务器 + 行协议客户端,并解释为什么不会粘包

本章知识导图

- §8.1 socket 三件套 vs std::net 三类型
- §8.2 客户端:connect 与读写
- §8.3 服务端:bind/accept 循环与每连接一线程
- §8.4 超时与半关闭:别让连接"永久卡死"
- §8.5 粘包与拆包:TCP 字节流的正确食用方式
- §8.6 UDP:send_to 与 recv_from
- §8.7 HTTP:request 客户端与最小服务端
- 速查表 → 最小必会清单 → 自测题 → 章末实验

§8.1 socket 三件套 vs std::net 三类型

8.1.1 C 的初始化仪式

```
// C:一个监听套接字的完整仪式
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

int fd = socket(AF_INET, SOCK_STREAM, 0);          // 1. 建套接字
if (fd < 0) { perror("socket"); return -1; }

int opt = 1;
setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof opt); // 2. 端口复用

struct sockaddr_in addr = {0};
addr.sin_family = AF_INET;
addr.sin_port = htons(8080);                      // 3. 网络字节序(大端)
addr.sin_addr.s_addr = INADDR_ANY;

if (bind(fd, (struct sockaddr *)&addr, sizeof addr) < 0) { perror("bind"); return -1; }
if (listen(fd, 128) < 0) { perror("listen"); return -1; } // 4. 开始监听

// 之后才能 accept(...) — 每一步都要查错,端口字节序要人肉 htons
```

8.1.2 Rust 的压缩形态

```
// Rust:bind + listen 一步到位,端口/地址自动处理字节序
use std::net::TcpListener;

fn main() -> std::io::Result<()> {
    // 绑定 0.0.0.0:8080(INADDR_ANY),失败即返回错误
    let listener = TcpListener::bind("0.0.0.0:8080")?;
    println!("监听中:{}", listener.local_addr()?);
    // 套接字随 listener 离开作用域自动关闭(RAII)
    Ok(())
}
```

对照:地址字符串(含端口)交给 ToSocketAddrs 解析,字节序由类型处理,没有 htons/ntohs、没有 SO_REUSEADDR 样板——标准库把"常见正确做法"做进了构造器。C 的 socket/fd 生命周期靠人记得 close,Rust 的 TcpListener/TcpStream 是 RAII 值,作用域结束即关闭。

C / POSIX	Rust std::net
socket() + setsockopt + bind + listen	TcpListener::bind("addr:port")? 一步完成
htons / ntohs 字节序转换	类型自动处理(ToSocketAddrs 解析)
accept() 返回 fd	listener.accept()? -> (TcpStream, SocketAddr)
connect()	TcpStream::connect("addr:port")?
send / recv / read / write	stream.read / write / read_exact / write_all
close(fd)	Drop 自动关闭(RAII)
errno / WSAGetLastError	io::Result 携带完整错误
fd 泄漏 / double-close	类型系统排除

§8.2 客户端:connect 与读写

```
use std::io::{Read, Write};
use std::net::TcpStream;

fn main() -> std::io::Result<()> {
    // 连接(对照 connect(2));失败信息含原因:拒绝、超时、无路由...
    let mut stream = TcpStream::connect("127.0.0.1:8080")?;
    println!("已连接:{}", stream.peer_addr()?);

    // 发送:write_all 保证整段发出(对照 send 返回值的循环检查)
    stream.write_all(b"GET / HTTP/1.1\r\nHost: 127.0.0.1\r\n\r\n")?;

    // 接收:循环读到 EOF(服务端关闭写方向后 read 返回 Ok(0))
    let mut buf = [0u8; 1024];
    let mut total = 0usize;
    loop {
        let n = stream.read(&mut buf)?;
        if n == 0 {
            break; // EOF:对端关闭
        }
        total += n;
        if let Ok(text) = std::str::from_utf8(&buf[..n]) {
            print!("{text}");
        }
    }
    println!("\n共接收 {total} 字节");
    Ok(())
}
```

对照要点:C 的 send/recv 返回值语义直接迁移

read 返回 0 = 对端关闭(EOF),负数 = 错误;write_all 封装了"部分写入要循环"的麻烦。C 程序员最常犯的"recv 只收了一半就当完整消息",在 Rust 里依然要自己处理——这正是 §8.5 的主题。唯一改变的是:错误不再是负数和 errno,而是 Result。

§8.3 服务端:bind/accept 循环与每连接一线程

```
use std::io::{BufRead, BufReader, Write};
use std::net::{TcpListener, TcpStream};
use std::thread;

/// 处理单个客户端连接:逐行读,原样写回(行协议回声)
fn handle_client(mut stream: TcpStream) -> std::io::Result<()> {
    // 读需要借用 stream,写需要可变借用:先克隆一份
    let mut reader = BufReader::new(stream.try_clone()?);

    for line in reader.lines() {
        let line = line?;
        if line == "quit" {
            break;
        }
        // 写回"回声:xxx";换行补上,保持行协议
        writeln!(stream, "回声:{line}")?;
    }
    println!("连接关闭:{}", stream.peer_addr()?);
    Ok(())
}

fn main() -> std::io::Result<()> {
    let listener = TcpListener::bind("127.0.0.1:8080")?;
    println!("服务端就绪,等待连接...");

    // 对照 C 的 while(1) { accept(); 处理; }:Rust 用迭代器 + 线程
    for stream in listener.incoming() {
        let stream = stream?;           // accept 失败:打印后继续等
        println!("新连接:{}", stream.peer_addr()?);

        // 每连接一个线程;stream 被 move 进闭包(所有权转移)
        thread::spawn(move || {
            if let Err(e) = handle_client(stream) {
                eprintln!("处理连接出错:{e}");
            }
        });
    }
    Ok(())
}
```

8.3.1 错误处理的对照

C 的 `accept` 失败(`errno=EMFILE` 等)要么终止要么靠全局 `errno` 判断;Rust 里 `stream = stream?` 一旦失败整个服务退出——生产上通常要"失败继续等"。上述代码中 `stream?` 直接传播,体现的是"教学清晰";生产模式是把 `accept` 的错误也打印后 `continue`(如下):

```
fn main() -> std::io::Result<()> {
    let listener = TcpListener::bind("127.0.0.1:8080");
    for stream in listener.incoming() {
        match stream {
            Ok(stream) => {
                thread::spawn(move || {
                    if let Err(e) = handle_client(stream) {
                        eprintln!("处理连接出错:{e}");
                    }
                });
            }
            Err(e) => eprintln!("accept 失败(继续监听):{e}"),
        }
    }
    Ok(())
}
```

⚠ 易错点 1:同一 TcpStream 上"边读边写"要 try_clone

BufReader::new(stream) 会把 stream 的读借用拿走;再想 writeln!(stream) 就"双重借用"编译失败。标准做法:stream.try_clone() 复制一份句柄,读用克隆、写用原柄。注意 try_clone 是操作系统层面的 dup——两个句柄共享同一个连接,不违反任何规则。C 里同样可以 dup(fd),只是没人告诉你这么用。

§8.4 超时与半关闭:别让连接永久卡死

```
use std::io::{Read, Write};
use std::net::TcpStream;
use std::time::Duration;

fn main() -> std::io::Result<()> {
    let mut stream = TcpStream::connect("10.0.0.5:22")?;

    // 读超时:5 秒内无数据到达则 Err(TimedOut)(对照 SO_RCVTIMEO)
    stream.set_read_timeout(Some(Duration::from_secs(5)))?;
    // 写超时:防止对端不读导致发送缓冲区塞满卡死(对照 SO_SNDTIMEO)
    stream.set_write_timeout(Some(Duration::from_secs(5)))?;

    stream.write_all(b"hello")?;

    // 半关闭:只关写方向,还能继续读(对照 shutdown(fd, SHUT_WR))
    // 意义:通知对端"我说完了",对端读到 EOF 后可以收尾
    stream.shutdown(std::net::Shutdown::Write)?;

    let mut buf = [0u8; 256];
    loop {
        match stream.read(&mut buf) {
            Ok(0) => break, // 对端也关闭了
            Ok(n) => println!("收到:{}", String::from_utf8_lossy(&buf[..n])),
            Err(e) if e.kind() == std::io::ErrorKind::TimedOut => {
                eprintln!("读超时,放弃");
                break;
            }
        }
        Err(e) => return Err(e),
    }
}
Ok(())
}
```

超时是生产网络的生存技能:无超时的阻塞读,对端崩溃后连接会挂到系统超时(几分钟),日志里全是"僵尸连接"。任何面向不可信对端的 read,都应该有超时。C 的 SO_RCVTIMEO 语义一致,但 Rust 把超时变成了每个流的显式设置,并区分错误种类(TimedOut 可以单独捕获)。

§8.5 粘包与拆包:TCP 字节流的正确食用方式

8.5.1 问题的本质

TCP 是字节流,不是消息流。发送方 write("你好") 两次,接收方可能一次 read 就拿到 "你好你好"(粘包),也可能两次 read 各拿一半(拆包)。这是内核缓冲与分段的物理事实,任何语言都一样。C 的解决方案是"应用层协议":要么每条消息以换行结束(行协议),要么头部带长度(长度前缀)。Rust 不改变这个事实,只把工具做得更顺手。

8.5.2 方案一:行协议(文本消息)

```
// 服务端:按行读,天然拆包;写回时补 \n,天然定界
use std::io::{BufRead, BufReader, Write};
use std::net::TcpListener;

fn main() -> std::io::Result<()> {
    let listener = TcpListener::bind("127.0.0.1:9000")?;
    for stream in listener.incoming() {
        let mut stream = stream?;
        let mut reader = BufReader::new(stream.try_clone()?);

        // lines() 内部处理"半行/多行"的切分—粘包拆包的第一道防线
        for line in reader.lines() {
            let line = line?;
            println!("收到消息:{}", line.trim_end());
            writeln!(stream, "已收到:{}", line)?;    // 响应同样以 \n 结尾
        }
    }
    Ok(())
}
```


8.5.3 方案二:长度前缀(二进制消息)

```
use std::io::{Read, Write};
use std::net::TcpStream;

// 协议:4 字节大端长度 + 正文
fn send_frame(stream: &mut TcpStream, payload: &[u8]) -> std::io::Result<()> {
    let len = (payload.len() as u32).to_be_bytes(); // 对照 htonl
    stream.write_all(&len)?; // 长度在前
    stream.write_all(payload)?; // 正文在后
    Ok(())
}

fn recv_frame(stream: &mut TcpStream) -> std::io::Result<Vec<u8>> {
    // 读 4 字节头:read_exact 保证整帧头(拆包时自动等齐)
    let mut head = [0u8; 4];
    stream.read_exact(&mut head)?;
    let n = u32::from_be_bytes(head) as usize; // 对照 ntohl
    if n > 1024 * 1024 { // 防炸弹:超限直接拒绝
        return Err(std::io::Error::new(
            std::io::ErrorKind::InvalidData, "帧过长"));
    }
    let mut body = vec![0u8; n];
    stream.read_exact(&mut body)?; // 读齐 n 字节
    Ok(body)
}

fn main() -> std::io::Result<()> {
    let mut stream = TcpStream::connect("127.0.0.1:9000")?;
    send_frame(&mut stream, b"第一条");
    send_frame(&mut stream, b"第二条"); // 连续两条也不会粘
    let reply = recv_frame(&mut stream)?;
    println!("服务端回复:{}", String::from_utf8_lossy(&reply));
    Ok(())
}
```

⚠ 易错点 2:长度前缀不校验,是生产事故之源

帧头给出 4GB 长度,你 allocate 4GB 内存然后 read——这就是"内存炸弹"。读长度后必须立即与上限比较(如 1MB),超限直接报错或断开。这与 C 里检查 ntohl 后的长度同理,只是 C 程序员通常要吃过一次苦头才记得。

★ 重点:粘包拆包在 Rust 里的解法不变,但工具变了

拆包(半条消息)由 read_exact 与 BufReader 消化:read_exact 等齐为止,中间由 lines() 自动攒行。粘包由协议定界解决:行协议看 \n,长度前缀看头部。C 需要手写"攒缓冲 + 找边界"的循环;Rust 的 BufRead trait(第7章已见)把这个循环封装成标准接口,网络与文件共用一套——这正是"一个 io 模型打通文件与网络"的体现。

§8.6 UDP:send_to 与 recv_from

```
use std::net::UdpSocket;

fn main() -> std::io::Result<()> {
    // UDP:无连接。bind 后直接收发(对照 socket + bind + sendto/recvfrom)
    let socket = UdpSocket::bind("127.0.0.1:0"?); // 端口 0:系统分配
    socket.connect("127.0.0.1:9000"?);           // 可选:固定对端(简化 send)

    // 发送(对照 sendto):数据报原子性由系统保证,一次一报
    socket.send(b"UDP 测试包"?);

    // 接收(对照 recvfrom):recv_from 返回 (数据, 来源地址)
    let mut buf = [0u8; 2048];
    let (n, src) = socket.recv_from(&mut buf)?;
    println!("来自 {src}:{n}", String::from_utf8_lossy(&buf[..n]));
    Ok(())
}
```

```
// UDP 服务端:无 accept,循环 recv_from
use std::net::UdpSocket;

fn main() -> std::io::Result<()> {
    let socket = UdpSocket::bind("127.0.0.1:9000"?);
    let mut buf = [0u8; 2048];
    loop {
        let (n, src) = socket.recv_from(&mut buf)?;
        // 每个包独立处理:无连接、无会话(对照 DNS、NTP 等协议模型)
        println!("收到 {src} 的 {n} 字节");
        let reply = format!("回:{n}\n", String::from_utf8_lossy(&buf[..n]));
        socket.send_to(reply.as_bytes(), src)?; // 回复要带来源地址
    }
}
```

场景	TCP	UDP
连接状态	面向连接,三次握手	无连接,直接发报
边界	字节流(要协议定界)	数据报(系统保边界)
可靠性	重传/排序/流控	可能丢、可能乱序
Rust 类型	TcpStream / TcpListener	UdpSocket(收发一体)
典型用途	HTTP、数据库、长连接	DNS、NTP、实时音视频、探活

§8.7 HTTP:request 客户端与最小服务端

8.7.1 HTTP 客户端:request(阻塞版)

```
// Cargo.toml 依赖:request = { version = "0.11", features = ["blocking"] }
// 用 request 的 blocking 客户端:同步 API,与 C 的 libcurl 用法同构
use std::time::Duration;

fn main() -> Result<(), Box<dyn std::error::Error>> {
    // GET 请求(对照 curl 的最简用法)
    let resp = request::blocking::get("https://example.com/");
    println!("状态码:{}", resp.status());
    println!("响应头:content-type = {}", resp.headers()["content-type"]
        .to_str()?);

    let body = resp.text()?; // 正文作为 UTF-8 文本
    println!("正文长度:{} 字符", body.len());

    // POST + JSON(对照 libcurl 的 POST 配置)
    let client = request::blocking::Client::builder()
        .timeout(Duration::from_secs(10)) // 全局超时
        .build()?;
    let resp = client
        .post("https://httpbin.org/post")
        .json(&serde_json::json!({"name": "rust", "n": 42}))
        .send()?;
    println!("POST 状态码:{}", resp.status());
    Ok(())
}
```

8.7.2 最小 HTTP 服务端(≤40 行,零依赖)

```
// 零依赖最小 HTTP 服务端:只处理 GET,返回固定页面
use std::io::{BufRead, BufReader, Write};
use std::net::{TcpListener, TcpStream};

fn handle(mut stream: TcpStream) -> std::io::Result<()> {
    let mut reader = BufReader::new(stream.try_clone()?);
    // 请求行:GET /path HTTP/1.1(读一行即可演示)
    let mut request_line = String::new();
    reader.read_line(&mut request_line)?;
    println!("请求:{request_line.trim()}");

    // 构造响应:状态行 + 头 + 空行 + 正文
    let body = "<h1>你好,Rust 服务端!</h1>\n";
    let response = format!(
        "HTTP/1.1 200 OK\r\nContent-Type: text/html; charset=utf-8\r\nContent-Length: {}\r\nConnection: close\r\n\r\n{}",
        body.len(), body
    );
    stream.write_all(response.as_bytes())?;
    Ok(())
}

fn main() -> std::io::Result<()> {
    let listener = TcpListener::bind("127.0.0.1:8080")?;
    println!("HTTP 服务在 http://127.0.0.1:8080/ 运行");
    for stream in listener.incoming() {
        let stream = stream?;
        std::thread::spawn(move || {
            let _ = handle(stream); // 单请求即关:Connection: close
        });
    }
    Ok(())
}
// 浏览器或 curl http://127.0.0.1:8080/ 即可看到页面
```

工程建议:生产 HTTP 服务用框架,别裸写

上面的 40 行服务端是为了看懂"HTTP 就是带格式的 TCP 字节流"。生产环境请直接上生态框架(axum、actix-web 或 tide),它们处理路由、并发、中间件、请求体解析。但理解本节仍不可替代:遇到"响应格式不对"Content-Length 不一致"的疑难杂症,能回到字节流层面推理,才是网络工程师的本事。

8.7.3 TIME_WAIT 与端口复用

C 服务端在 bind 前要 setsockopt(SO_REUSEADDR),否则服务重启时 TIME_WAIT 状态(主动关闭方在 2×MSL 内保留端口)会让 bind 失败。Rust 标准库 TcpListener::bind 不暴露这个选项——在 Unix 上它默认不设置 SO_REUSEADDR,重启服务可能碰到 AddrInUse。需要该行为时,用 socket2 库显式设置;或者把"主动关闭"留给对端(服务端先关连接时自己是主动方)。生产部署中,优雅重启(先关监听、等存量连接结束)是比 SO_REUSEADDR 更彻底的方案。

⚠ 易错点 3:对端崩溃后,read 不会立即返回

TCP 连接上对端进程崩溃,内核可能几十分钟后才发 RST;这期间的 read 会一直阻塞。生产客户端必须:① 设读写超时(\$8.4);② 需要"对端还活着吗"时用应用层心跳。C 的教训(SO_KEEPALIVE、select 超时)原样适用——区别是 Rust 把超时做成每个流的显式配置,忘了设就等着连接挂死,而错误种类(TimedOut)可以直接捕获分流。

本章速查表

主题	结论 / 写法
监听套接字	TcpListener::bind("0.0.0.0:8080")?(bind+listen 一步)
接受连接	listener.accept()? -> (TcpStream, SocketAddr);或 for s in listener.incoming()
客户端连接	TcpStream::connect("127.0.0.1:8080")?
发送	stream.write_all(bytes)?(保证全量;对照 send 循环)
接收	read 返回 Ok(0) = EOF;定长读用 read_exact
边读边写	stream.try_clone()? 后,BufReader 用克隆,写用原柄
读/写超时	set_read_timeout / set_write_timeout(Some(Duration));TimedOut 单捕
半关闭	stream.shutdown(Shutdown::Write)?(对照 shutdown(2))
粘包拆包	TCP 是字节流;行协议用 lines(),二进制用"长度前缀 + read_exact"
长度上限	读长度后立即校验上限,防内存炸弹
UDP 收发	UdpSocket::bind;send_to / recv_from -> (n, src)
HTTP 客户端	reqwest(blocking 特性):Client::builder().timeout().build()
HTTP 服务端	手写 ≤40 行(教学)或 axum/actix-web(生产)
套接字关闭	RAII:作用域结束自动关闭;无 fd 泄漏

最小必会清单

- 能写出 TcpListener::bind + incoming 循环的服务端骨架,错误全程 Result
- 能写出 TcpStream::connect + write_all + read 循环的客户端骨架,识别 EOF
- 知道边读边写要 try_clone,并能解释它为什么是 dup 而非复制数据
- 能说出 TCP 无消息边界,粘包拆包的本质;能用行协议或长度前缀解决
- 会写 read_exact 读定长帧头 + 长度校验防炸弹
- 知道读写超时的设置方法与 TimedOut 的捕获方式
- 能用 UdpSocket 收发,并说出与 TCP 在边界、可靠性上的差异
- 会用 reqwest blocking 客户端发 GET/POST,处理状态码与正文
- 能逐字段解释 HTTP 响应的"状态行 + 头 + 空行 + 正文"
- 知道 TIME_WAIT 与 AddrInUse 的关系,以及生产上的优雅重启思路

自测题

Q 1. TcpListener::bind("127.0.0.1:8080") 一步完成了 C 的哪几步?省略了什么?

✓ 参考答案

完成:socket() + bind() + listen()(以及地址解析与字节序处理)。未完成:SO_REUSEADDR 等 setsockopt 选项(标准库不暴露,需要时用 socket2);accept 循环与每连接处理(那是业务代码)。省略掉的是"必须正确的仪式",留下的是"业务可选的调优"。

Q 2. read 返回 Ok(0) 意味着什么?在 C 里对应什么?哪些场景会导致它迟迟不来?

✓ 参考答案

Ok(0) = EOF:对端已关闭连接(或 shutdown 了写方向),不会再发数据。对应 C 的 recv 返回 0。迟迟不来的场景:对端进程崩溃但连接未感知(内核还没发 RST)、对端只是不发送数据(长连接挂起)、网络故障。所以生产读循环要么有超时,要么有应用层心跳。

Q 3. 为什么 TCP 会粘包,而 UDP 不会?粘包能"关掉"吗?

✓ 参考答案

TCP 是面向字节流的:内核把多次 write 的数据按需合并/切分放进报文,接收方看到的只是字节序列,没有"消息"概念,所以多段数据可能粘在一次 read 里(粘包)或一段数据分多次 read(拆包)。UDP 以数据报为单位收发,系统保证一次 send 对应一次 recv 的边界。粘包是 TCP 的本质特性,无法关闭,只能由应用层协议定界(行协议/长度前缀)解决。

Q 4. 帧协议里,为什么读长度之后必须校验上限?

✓ 参考答案

长度字段来自网络,由对端控制。若不校验,恶意或损坏的帧头给出 4GB,程序会申请 4GB 内存并阻塞等待数据——内存炸弹与资源耗尽攻击。对策:与协议上限比较,超限立即报错/断开。这与 C 的教训一致,但 read_exact + 显式上限让防线写起来只有三行。

Q 5. 每连接一线程的服务端,线程里为什么用 move 闭包?如果不用会怎样?

✓ 参考答案

stream 在 main 的循环里创建,线程闭包要长期持有它。move 把 stream 的所有权转移进新线程——这是 Rust 保证"多线程不同时访问同一所有权"的机制(第9章详述)。不用 move,闭包只能借用循环变量,而循环会覆盖它,编译错误(borrowed value does not live long enough)——编译器强迫你做出正确选择。

Q 6. 服务端重启时报 Address already in use,原因是什么?Rust 怎么处理?

✓ 参考答案

旧进程主动关闭连接时进入 TIME_WAIT 状态(持续约 $2 \times \text{MSL}$),端口未完全释放;而 Rust 标准库的 bind 默认不设置 SO_REUSEADDR,于是新进程 bind 失败。对策:① 用 socket2 设置 SO_REUSEADDR(与 C 相同);② 优雅重启:

先关闭监听、等待存量连接自然结束再退出(生产首选);③ 改端口。

章末实验题:回声服务器与行协议客户端

实验 8:回声服务器与行协议客户端

实验目标:实现一个行协议回声系统:服务端接受多客户端,逐行读入并原样回复;客户端连接后连续发送若干行消息,校验回复。

实验要求:

- 任务 1:服务端 bind + incoming 循环,每连接一线程(覆盖:服务端骨架 §8.3)
- 任务 2:用 BufReader + lines() 读行,回复加前缀并补换行(覆盖:行协议 §8.5)
- 任务 3:客户端连接、循环发送、逐行读回并校验(覆盖:客户端 §8.2)
- 任务 4:设置读超时,测试"服务端不回复"时的超时路径(覆盖:超时 §8.4)
- 任务 5:客户端支持命令行指定"消息条数",批量发送模拟粘包压力(覆盖:粘包理解 §8.5)
- 加分:服务端统计每个连接的消息数,连接结束时打印(覆盖:每连接状态)

运行方式:两个终端:cargo run --bin server(或先启动服务端),再 cargo run --bin client -- 5(发送 5 条)。

第一步 **一步:**cargo new echo 后,建 src/bin/server.rs 与 src/bin/client.rs 两个入口

第二步 **二步:**先写服务端:bind、打印地址、incoming 循环 + 线程 + 行协议回声

第三步 **三步:**写客户端:connect、循环发送 "消息 N"、逐行读回并断言前缀

第四步 **四步:**加超时设置,故意停掉服务端,观察客户端超时退出而不是永久挂起

第五步 **五步:**把客户端改成"一次写出全部消息"(write_all 一段含多行的数据),观察服务端能否逐行还原——验证 BufReader 拆包

参考答案要点

```
// ===== 实验 8:回声服务器与行协议客户端 =====
// 覆盖知识点:服务端骨架与每连接线程(§8.3)、行协议拆包(§8.5)、
// 客户端读写(§8.2)、超时(§8.4)、粘包验证(§8.5)

// ---- src/bin/server.rs ----
use std::io::{BufRead, BufReader, Write};
use std::net::{TcpListener, TcpStream};
use std::thread;

fn handle_client(mut stream: TcpStream) -> std::io::Result<()> {
    let mut reader = BufReader::new(stream.try_clone()?);
    let mut count = 0u32;
    for line in reader.lines() {
        let line = line?;
        count += 1;
        if line == "quit" {
            break;
        }
        // 回声:原样 + 前缀;补 \n 维持行协议
        writeln!(stream, "echo:{line}")?;
    }
    // 加分:连接结束时统计
    println!("连接 {} 关闭,共处理 {} 条消息",
        stream.peer_addr()?);
    Ok(())
}

fn main() -> std::io::Result<()> {
    let listener = TcpListener::bind("127.0.0.1:8080");
    println!("回声服务端监听 {}", listener.local_addr()?);
    for stream in listener.incoming() {
        match stream {
            Ok(stream) => {
                thread::spawn(move || {
                    if let Err(e) = handle_client(stream) {
                        eprintln!("处理连接出错:{e}");
                    }
                });
            }
            Err(e) => eprintln!("accept 失败(继续监听):{e}"),
        }
    }
    Ok(())
}

// ---- src/bin/client.rs ----
use std::io::{BufRead, BufReader, Write};
use std::net::TcpStream;
use std::time::Duration;

fn main() -> std::io::Result<()> {
    // 消息条数来自命令行
    let n: u32 = std::env::args()
        .nth(1).unwrap_or_else(|| "3".into())
        .parse().expect("条数必须是数字");

```



```

let mut stream = TcpStream::connect("127.0.0.1:8080");
// 任务 4:读超时 5 秒
stream.set_read_timeout(Some(Duration::from_secs(5)));
let mut reader = BufReader::new(stream.try_clone()?);

// 任务 5:批量发送——一次 write_all 含多行,验证服务端逐行还原
let mut batch = String::new();
for i in 1..=n {
    batch.push_str(&format!("消息 {i}\n"));
}
stream.write_all(batch.as_bytes())?;

// 逐行读回校验
for i in 1..=n {
    let mut line = String::new();
    match reader.read_line(&mut line) {
        Ok(0) => {
            eprintln!("服务端提前关闭(第 {i} 条)");
            std::process::exit(1);
        }
        Ok(_) => {
            let expect = format!("echo:消息 {i}");
            assert!(line.trim() == expect,
                "第 {i} 条回复不匹配:got '{}' want '{expect}'",
                line.trim());
            println!("✓ {line}", );
        }
        Err(e) if e.kind() == std::io::ErrorKind::TimedOut => {
            eprintln!("读超时:服务端无响应(第 {i} 条)");
            std::process::exit(2);
        }
        Err(e) => return Err(e),
    }
}
// 收尾:发 quit 让服务端优雅关闭该连接
stream.write_all(b"quit\n")?;
println!("全部 {n} 条校验通过");
Ok(())
}

```

设计说明:客户端一次写出整批消息,是验证"拆包"的标准手法——TCP 可能把这批数据切成任意大小的段,服务端的 `lines()` 必须逐行还原,任何一行错位都会让断言失败。把断言换成 `println` 再跑一遍,你会看到"粘包"其实从没被消除,只是被协议层正确地重建了。超时分支用 `ErrorKind::TimedOut` 单独捕获——这正是"错误是类型"在生产中的价值:C 里同一套 `errno` 要区分五六种含义,Rust 直接按 `kind` 分流。

下一章预告:第9章 并发与异常处理——C 的 `errno` 与 `pthread` 对应 Rust 的 `Result`? 与 `std::thread`; `Send`/`Sync`、`Mutex`/原子、通道与 `tokio` 异步入门。